

责任链模式重构复杂业务场景

文章开篇，抛出一个老生常谈的问题，**学习设计模式有什么作用？**

设计模式主要是为了应对代码的复杂性，让其满足开闭原则，提高代码的扩展性。

另外，学习的设计模式 **一定要在业务代码中落实**，只有理论没有真正实施，是无法真正掌握并且灵活运用设计模式的。

这篇文章主要说 **责任链设计模式**，认识此模式是在读 Mybatis 源码时，Interceptor 拦截器主要使用的就是责任链，当时读过后就留下了很深的印象（内心 OS：还能这样玩）。

文章先从基础概念说起，另外分析一波 Mybatis 源码中是如何运用的，最后按照 "习俗"，设计一个真实业务场景上的应用。

什么是责任链模式

举个例子，SpringMvc 中可以定义拦截器，并且可以定义多个。当一个用户发起请求时，顺利的话请求会经过所有拦截器，最终到达业务代码逻辑，SpringMvc 拦截器设计就是使用了责任链模式。

为什么说顺利的话会经过所有拦截器？因为请求不满足拦截器自定义规则会被打回，但这并不是责任链模式的唯一处理方式，继续往下看。

在责任链模式中，多个处理器（参照上述拦截器）依次处理同一个请求。一个请求先经过 A 处理器处理，然后再把请求传递给 B 处理器，B 处理器处理完后再传递给 C 处理器，以此类推，形成一个链条，链条上的每个处理器 **各自承担各自的处理职责**。

责任链模式中多个处理器形成的处理器链在进行处理请求时，有两种处理方式：

1. 请求会被 **所有的处理器都处理一遍，不存在中途终止的情况**，这里参照 MyBatis 拦截器理解。
2. 二则是处理器链执行请求中，某一处理器执行时，**如果不符合自制定规则的话，停止流程，并且剩下未执行处理器就不会被执行**，大家参照 SpringMvc 拦截器理解。

这里通过代码的形式对两种处理方式作出解答，方便读者更好的理解。首先看下第一种，请求会经过所有处理器执行的情况。

```
interface IHandler {
    void handle();
}

class HandlerA implements IHandler {
    @Override
    public void handle() { System.out.println("执行 HandlerA"); }
}

class HandlerB implements IHandler {
    @Override
    public void handle() { System.out.println("执行 HandlerB"); }
}

class HandlerChain {
    List<IHandler> handlerList = new ArrayList<>();

    public void addHandler(List<IHandler> handlers) {
        handlerList.addAll(handlers);
    }

    public void handle() {
        handlerList.forEach(IHandler::handle);
    }
}
```

`IHandler` 负责抽象处理器行为，`handle()` 则是不同处理器具体需要执行的方法，`HandlerA`、`HandlerB` 为具体需要执行的处理器类，`HandlerChain` 则是将处理器串成一条链执行的处理器链。

```
public class ChainApplication {
    public static void main(String[] args) {
        HandlerChain handlerChain = new HandlerChain();
        handlerChain.addHandler(Lists.newArrayList(new HandlerA(), new HandlerB()));
        handlerChain.handle();
        /**
         * 程序执行结果:
         * HandlerA打印: 执行 HandlerA
         * HandlerB打印: 执行 HandlerB
         */
    }
}
```

这种责任链执行方式会将所有的 **处理器全部执行一遍**，不会被打断。Mybatis 拦截器用的正是此类型，这种类型 **重点在对请求过程中的数据或者行为进行改变**。

而另外一种责任链模式实现，则是会对请求有阻断作用，阻断产生的前置条件是在处理器中自定义的，代码中的实现较简单，读者可以联想 SpringMvc 拦截器的实现流程。

```

interface IHandler { boolean handle(); }

class HandlerA implements IHandler {
    @Override
    public boolean handle() {
        boolean result = true;
        System.out.println("执行 HandlerA"); // 根据自定义规则看是否符合, 符合返回 true 不符合返回 false
        return result;
    }
}

class HandlerB implements IHandler {
    @Override
    public boolean handle() {
        boolean result = true;
        System.out.println("执行 HandlerB"); // 根据自定义规则看是否符合, 符合返回 true 不符合返回 false
        return result;
    }
}

class HandlerChain {
    List<IHandler> handlerList = new ArrayList<>();

    public void addHandler(List<IHandler> handlers) { handlerList.addAll(handlers); }

    public boolean handle() {
        boolean resultFlag = true;
        for (IHandler each : handlerList) {
            if (!each.handle()) {
                resultFlag = false; break;
            }
        }
        return resultFlag;
    }
}

```

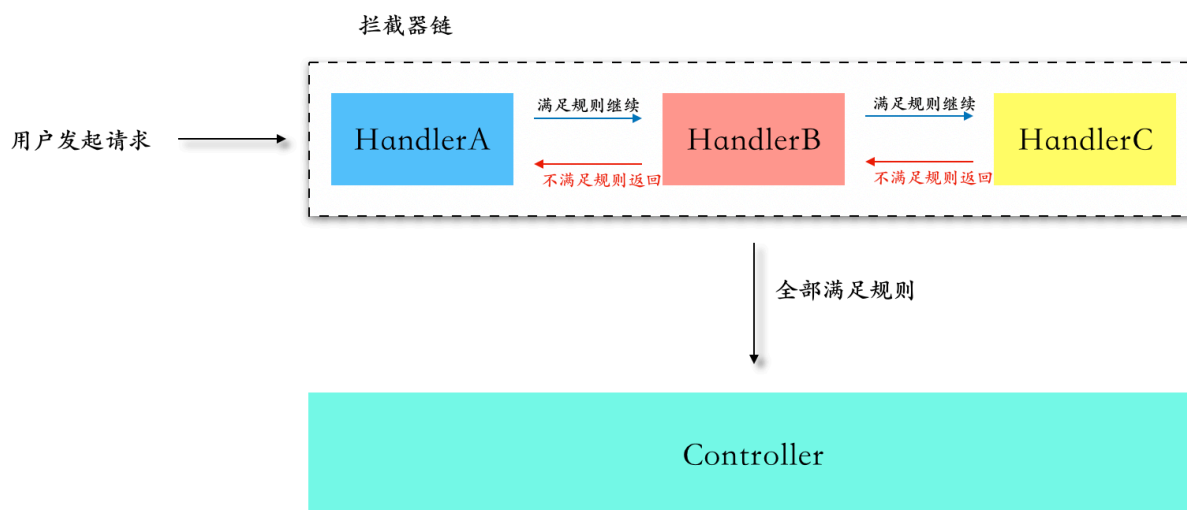
根据代码看的出来, 在每一个 IHandler 实现类中会返回一个布尔类型的返回值, 如果返回布尔值为 false, 那么责任链发起类会中断流程, 剩余处理器将不会被执行。就像我们定义在 SpringMvc 中的 Token 拦截器, 如果 Token 失效就不能继续访问系统, 处理器将请求打回。

```

public class ChainApplication {
    public static void main(String[] args) {
        HandlerChain handlerChain = new HandlerChain();
        handlerChain.addHandler(Lists.newArrayList(new HandlerA(), new HandlerB()));
        boolean resultFlag = handlerChain.handle();
        if (!resultFlag) {
            System.out.println("责任链中处理器不满足条件");
        }
    }
}

```

读者可以自己在 IDEA 中实现两种不同的责任链模式, 对比其中的不同, 设想下业务中真实的应用场景, 又或者可以跑 SpringBoot 项目, 创建多个拦截器来佐证文中的说辞。



本章节介绍了责任链设计模式的具体语义，以及不同责任链实现类型代码举例，并以 Mybatis、SpringMvc 拦截器为参照点，介绍各自不同的代码实现以及应用场景。

责任链业务场景设计

趁热打铁，本小节对使用的真实业务场景进行举例说明。假设业务场景是这样的，我们 **系统处在一个下游服务**，因为业务需求，系统中所使用的 **基础数据需要从上游中台同步到系统数据库**。

基础数据包含了很多类型数据，虽然数据在中台会有一定验证，但是 **数据只要是人为录入就极可能存在问题，遵从对上游系统不信任原则**，需要对数据接收时进行一系列校验。

最初是要进行一系列验证原则才能入库的，后来因为工期问题只放了一套非空验证，趁着春节期间时间还算宽裕，把这套验证规则骨架放进去。

从我们系统的接入数据规则而言，个人觉得需要支持以下几套规则：

1. **必填项校验**，如果数据无法满足业务所必须字段要求，数据一旦落入库中就会产生一系列问题。
2. **非法字符校验**，因为数据如何录入，上游系统的录入规则是什么样的我们都不清楚，这一项规则也是必须的。

3. **长度校验**，理由同上，如果系统某字段长度限制 50，但是接入来的数据 500 长度，这也会造成问题。

为了让读者了解业务嵌入责任链模式的前因，这里列举了三套校验规则，当然真实中可能不止这三套。但是 **一旦将责任链模式嵌入数据同步流程**，就会 **完全符合文初所提的开闭原则，提高代码的扩展性**。

本案例设计模式中的开闭原则通过 Spring 提供支持，后续添加新的校验规则就可以不必修改原有代码。

这里要再强调下，**设计模式的应用场景一定要灵活掌握**，只有这样才能在合适的业务场景合理运用对象的设计模式。

既然设计模式场景说过了，最后说一下需要达成的业务需求。将一个批量数据经过处理器链的处理，**返回出符合要求的数据分类**。

```
public interface VerifyHandler { // 定义验证接口
    List<Object> verify(List<Object> allObjList);
}

@Component // 非空字段校验处理器
public class EmptyVerifyHandler implements VerifyHandler {
    @Override
    public List<Object> verify(List<Object> allObjList) {
        // 将传入数据进行非空验证，返回符合正确数据
        System.out.println("🚀🚀🚀 进入 EmptyVerifyHandler");
        // xxx
        List<Object> trueList = allObjList;
        return trueList;
    }
}

@Component // 字段敏感词校验处理器
public class SexyVerifyHandler implements VerifyHandler {
    @Override
    public List<Object> verify(List<Object> allObjList) {
        // 将传入数据进行敏感词过滤，返回符合正确数据
        System.out.println("🚀🚀🚀 进入 SexyVerifyHandler");
        // xxx
        List<Object> trueList = allObjList;
        return trueList;
    }
}
```

定义顶级验证接口和一系列处理器实现类没什么难度，但是应该如何进行链式调用呢？

这一块代码需要有一定 Spring 基础才能理解，一起来看下 VerifyHandlerChain 如何将所有处理器串成一条链。

```
@Component
public class VerifyHandlerChain implements InitializingBean {

    @Autowired
    private ApplicationContext applicationContext;

    public final List<VerifyHandler> verifyHandlerList = new ArrayList();

    @Override
    public void afterPropertiesSet() throws Exception {
        // 获取 IOC 容器中类型为 VerifyHandler 的 Bean，也就是 EmptyVerifyHandler、SexyVerifyHandler
        Map<String, VerifyHandler> beansOfType = applicationContext.getBeansOfType(VerifyHandler.class);
        // 添加到校验集合中
        beansOfType.forEach((key, val) -> verifyHandlerList.add(val));
    }

    public List<Object> verify(List<Object> allObjList) {
        List<Object> resultList = null;
        for (VerifyHandler each : verifyHandlerList) {
            // 对数据展开处理器链的全部调用，如果过程中发现已无需要验证的数据，直接返回
            resultList = each.verify(allObjList);
            if (CollectionUtils.isEmpty(resultList)) {
                return resultList;
            }
        }
        return resultList;
    }
}
```

VerifyHandlerChain 处理流程如下：

1. 实现自 InitializingBean 接口，在对应实现方法中获取 IOC 容器中类型为 VerifyHandler 的 Bean，也就是 EmptyVerifyHandler、SexyVerifyHandler。
2. 将 VerifyHandler 类型的 Bean 添加到处理器链容器中。
3. 定义校验方法 verify()，对入参数据展开处理器链的全部调用，如果过程中发现已无需要验证的数据，直接返回。

这里使用 SpringBoot 项目中默认测试类，来测试一下如何调用。

```
@SpringBootTest
class ChainApplicationTests {

    @Autowired
    private VerifyHandlerChain verifyHandlerChain;

    @Test
    void contextLoads() {
```

```
List<Object> verify = verifyHandlerChain.verify(Lists.newArrayList("马丁玩编程",
"马丁"));
System.out.println(verify);
}
}
```

这样的话，如果客户或者产品提校验相关的需求时，我们只需要实现 VerifyHandler 接口新建个校验规则实现类就 OK 了，这样符合了设计模式的原则：**满足开闭原则，提高代码的扩展性。**

熟悉之前作者写过设计模式的文章应该知道，**强调设计模式重语意，而不是具体的实现过程。**所以，你看这个咱们这个校验代码，把责任链两种模式结合了使用。

上面的代码只是示例代码，实际业务中的实现要比这复杂很多，比如：

1. **如何定义处理器的先后调用顺序。**比如说某一个处理器执行时间很长并且过滤数据很少，所以希望把它放到最后面执行。
2. 这是为当前业务的所有数据类型进行过滤，**如何自定义单个数据类型过滤。**比如你接入学生数据，学号有一定校验规则，这种处理器类肯定只适合单一类型。

还有很多的业务场景，**所以设计模式强调的应该是一种思想，而不是固定的代码写法**，需要结合业务场景灵活变通。

责任链模式的好处

一定要使用责任链模式么？不使用能不能完成业务需求？

回答是肯定可以，设计模式只是帮助减少代码的复杂性，让其满足开闭原则，提高代码的扩展性。如果不使用同样可以完成需求。

如果不使用责任链模式，上面说的真实同步场景面临两个问题：

1. 如果把上述说的代码逻辑校验规则写到一起，**毫无疑问这个类或者说这个方法函数奇大无比。**减少代码复杂性一贯方法是：**将大块代码逻辑拆分成函数，将大类拆分成小类**，是应对代码复杂性的常用方法。如果此时说：可以把不同的校验规则拆分成不同的函数，不同的类，这样不也可以满足减少代码复杂性的要求么。这样拆分是能解决代码复杂性，但是这样就会面临第二个问题。
2. 开闭原则：**添加一个新的功能应该是，在已有代码基础上扩展代码，而非修改已有代码。**大家设想一下，假设你写了三套校验规则，运行过一段时间，这时候领导让加第四套，是不是要在

原有代码上改动。

综上所述，在合适的场景运用适合的设计模式，能够让代码设计复杂性降低，变得更为健壮。朝更远的说也能让自己的编码设计能力有所提高，告别被人吐槽的烂代码...

Mybatis Interceptor 底层实现

上面说了那么多，框架底层源码是怎么设计并且使用责任链模式的？之前在看 Mybatis 3.4.x 源码时了解到 Interceptor 底层实现就是责任链模式，这里和读者分享 Interceptor 具体实现。

开门见山，直接把视线聚焦到 Mybatis 源码，版本号 3.4.7-SNAPSHOT。

```
/**
 * @author Clinton Begin
 */
public class InterceptorChain {

    private final List<Interceptor> interceptors = new ArrayList<Interceptor>();

    public Object pluginAll(Object target) {
        for (Interceptor interceptor : interceptors) {
            target = interceptor.plugin(target);
        }
        return target;
    }

    public void addInterceptor(Interceptor interceptor) {
        interceptors.add(interceptor);
    }

    public List<Interceptor> getInterceptors() {
        return Collections.unmodifiableList(interceptors);
    }

}
```

熟悉么？是不是和我们上面用到的责任链模式差不太多，有处理器集合 interceptors，有添加处理器方法。

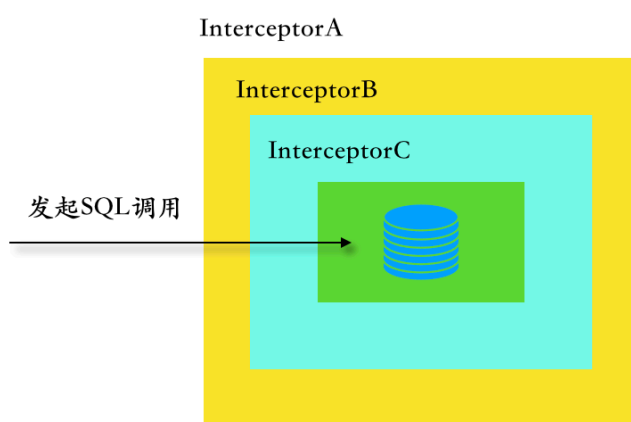
Mybatis Interceptor 不仅用到了责任链，还用到了动态代理，服务于 Mybatis 四大 "护教法王"，在创建对象时通过动态代理和责任链相结合组装而成插件模块。

1. ParameterHandler。
2. ResultSetHandler。
3. StatementHandler。

4. Executor。

使用过 Mybatis 的读者应该知道，查询 SQL 的分页语句就是使用 Interceptor 实现，比如市场上的 PageHelper、Mybatis-Plus 分页插件再或者我们自实现的分页插件（应该没有项目组使用显示调用多条语句组成分页吧）。

拿查询语句举例，如果定义了多个查询相关的拦截器，会先经过拦截器的代码加工，所有的拦截器执行完毕后会走真正查询数据库操作。



扯的话就扯远了，能够知道如何用、在哪用就可以了。通过 Interceptor 也能知道一点，**想要读框架源码，需要一定的设计模式基础**。如果对责任链、动态代理不清楚，那么就不能理解这一块的精髓。

文末结语

文章通过图文并茂的方式帮助大家理解责任链设计模式，在两种类型示例代码以及举例实际业务场景下，相信小伙伴已经掌握了如何在合适的场景使用责任链设计模式。

看完文章后可以结合 Mybatis、SpringMvc 拦截器更深入掌握责任链模式的应用场景以及使用手法。另外可以结合项目中实际业务场景灵活使用，相信真正使用后的你会对责任链模式产生更深入的了解。

参考文章：《设计模式之美：职责链模式》

更新: 2023-07-25 16:01:02

原文: <<https://www.yuque.com/magestack/12306/xxdps9bcasca7cyv>>